

Reflection Questions - Svar

How many tests are needed to find all bugs in the examples? Do you think this is enough testing for these problems? Too much?

Password (Practice: Write and Run Tests)

Svar: 9 tester behövs för att hitta alla 10 buggar.

- Test 1-6 och 8-9: Fångar var sin unik bugg (8 tester)
- Test 7: Fångar både BugToShortPassword OCH BugWrongMessage (1 test för 2 buggar)
- Test 11: Extra validering för typkontroll (fångar ingen bugg men testar edge case)

Är detta tillräckligt? Delvis. För att hitta buggar är det tillräckligt, men för robust produktion skulle jag lägga till:

- Fler boundary tests (exakt 12 tecken, mycket långa lösenord)
- Special characters och unicode
- Siffra på olika positioner

För mycket? Nej, Test 11 är värdefullt för input validering även om det inte fångar någon av de givna buggarna.

SSNHelper (Practice: Write Mocked Tests)

Svar: 6 tester behövs för att hitta alla 7 buggar.

- Test 1: Fångar BuggyAllowMonth0 OCH BuggyWrongLength
- Test 2-6: Fångar var sin unik bugg (5 tester)

Är detta tillräckligt? Ja, för dessa buggar. Men för verklig validering av svenska personnummer skulle jag lägga till:

- Test för giltiga datum (t.ex. inte 31 februari)
- Test för moderna personnummer (2000+)
- Test för samordningsnummer

För mycket? Nej, varje test har unikt värde.

SwedishSocialSecurityNumber (Practice: Write Mocked Tests)

Svar: 4 tester behövs för att hitta alla 4 buggar.

Perfekt 1:1 mappning:

- Test 1: BuggyNoTrim
- Test 2: BuggyNoLenCheck
- Test 3: BuggyNoLuhn
- Test 4: BuggyWrongYear

Är detta tillräckligt? För de givna buggarna - ja. För produktion skulle jag lägga till tester för edge cases och integration med SSNHelper.

För mycket? Nej, optimal testsvit.

What are the missing tests you think would be good to add?

Password

1. Boundary testing för längd

- Test med exakt 12 tecken (minsta giltiga längd)
- Test med mycket långt lösenord (100+ tecken)

2. Number position testing

- Siffra först: "1validPassword"
- Siffra i mitten: "validPass1word"
- Flera siffror: "valid123Password"

3. Whitespace edge cases

- Endast spaces: " "
- Spaces i mitten: "valid Pass1word"
- Tab och newline tecken

4. Special characters

- Special tecken: "valid@Pass1word!"
- Unicode tecken: "välid1Påsswörd"

5. Hash collision testing

- Mer omfattande test med tusentals lösenord

SSNHelper

1. Date validation

- Ogiltiga datum: "850230-1234" (30 februari)
- "850431-1234" (31 april)

2. Modern dates

- 2000+ personnummer: "000101-1234"
- Future dates

3. Format variations

- Med/utan bindestreck
- 10-siffrig vs 12-siffrig

SwedishSocialSecurityNumber

1. Integration testing

- Test med riktig SSNHelper (inte mock)

2. Error message validation

- Verifiera exakta felmeddelanden

3. Edge cases

- Mycket lång input
 - Null/undefined input
-

What is good test data for this example and why?

Egenskaper för bra testdata

1. Edge Cases (Gränsfall)

Viktigt att testa gränser eftersom buggar ofta uppstår där:

- **Tom sträng** (' '): Minimal input
- **6 tecken** ('a1b2c3'): Just under minimum (hittar BugVeryShort)
- **11 tecken** ('0123456789a'): En tecken under minimum (hittar BugToShortPassword)
- **12 tecken**: Minsta giltiga längd
- **Mycket lång sträng**: Testar max capacity

2. Representative Data (Representativ data)

Normal användning:

- **Giltigt lösenord** ('validPassword1'): Normal use case
- **Lösenord utan siffra** ('longenoughpassword'): Testar number-validering
- **Lösenord med spaces** (' validPassword1 '): Testar trimming

3. Boundary Values för SSN

- **Dag 00**: Ogiltigt minimum (hittar BuggyAllowDay00)
- **Dag 31**: Giltigt maximum
- **Månad 00**: Ogiltigt minimum
- **Månad 13**: Ogiltigt maximum
- **Längd 11**: Precis under minimum

4. Invalid Input

- Null/undefined
- Fel datatyper
- Tomma strängar

Varför är detta bra testdata?

☒ **Täcker alla validators:** Längd, siffror, trimming, format, Luhn ☒ **Testar gränsvärden:** Där buggar oftast uppstår ☒ **Både giltiga och ogiltiga inputs:** Positiva och negativa testfall ☒ **Realistiska scenarios:** Representerar verklig användning ☒ **Edge cases:** Tom sträng, whitespace, extremvärden

Should private methods be tested? What are the pros and cons of having a "helper class" instead of private methods?

Ska privata metoder testas?

Svar: Generellt NEJ - testa INTE privata metoder direkt.

Varför inte?

1. **Inkapsling:** Privata metoder är implementation details som kan ändras
2. **Fragile tests:** Tester mot privata metoder blir beroende av intern implementation
3. **Public API:** Vi bör testa genom den publika API:n
4. **Refactoring:** Om vi testar privata metoder blir det svårt att refaktörera
5. **Test maintenance:** Interna ändringar bryter tester

I våra exempel

Password - Privata metoder `#isTooShort()`, `#containsNumber()`, `#simpleHash()` testas **indirekt**:

- Constructor-tester → använder alla tre metoder
- getPasswordHash() → testar #simpleHash indirekt
- isPasswordSame() → testar hash-jämförelse

SSNHelper - Publika metoder testas direkt utan problem.

När det MÖJLIGEN kan vara OK

- Komplex algoritm i privat metod som är kritisk
- Legacy code med mycket logik
- **Men då bör man överväga att extrahera till helper class istället!**

Pros och Cons: Helper Class vs Private Methods

Helper Class (t.ex. SSNHelper)

Pros: ☒ **Testbarhet:** Kan testas isolerat ☒ **Återanvändbarhet:** Kan användas av flera klasser ☒ **Separation of Concerns:** Tydlig ansvarsfördelning ☒ **Mocking:** Kan enkelt mockas i tester ☒ **Single Responsibility:** Varje klass har ett ansvar

Cons: ☒ **Fler klasser:** Mer kod att underhålla ☒ **Overhead:** Extra objekt att instansiera ☒ **Komplexitet:** Fler beroenden att hantera

Private Methods

Pros: ☒ **Enklare struktur:** Allt i en klass ☒ **Inkapsling:** Implementation dold ☒ **Mindre overhead:** Ingen extra klass ☒ **Snabbare utveckling:** Mindre boilerplate

Cons: ☒ **Svår att testa direkt:** Måste testas genom public API ☒ **Svår att återanvända:** Låst till en klass ☒ **Svår att mocka:** Kan inte ersättas i tester ☒ **Större klasser:** Risk för "God objects"

Bästa praxis

Använd Helper Class när:

- Logiken är komplex och värd att testa isolerat
- Logiken kan återanvändas
- Du vill kunna mocka beteendet
- Exempel: SSNHelper används av SwedishSocialSecurityNumber

Använd Private Methods när:

- Enkla utility-funktioner
- Implementation details som aldrig testas direkt
- Kod som bara används internt i klassen

Can all code be covered by code coverage? Why not?

Svar: NEJ, all kod kan inte alltid täckas av code coverage.

Kod som är svår/omöjlig att täcka

1. Error handling för system errors

```
try {  
    // Normal kod  
} catch (OutOfMemoryError e) {  
    // Svårt att simulera  
}
```

- Out of memory exceptions
- Disk full errors
- Network failures (om inte mockad)

2. Defensive programming

```
if (impossibleCondition) {  
    throw new Error('This should never happen')  
}
```

Kod som teoretiskt aldrig kan nås men finns för säkerhet.

3. Dead code

```
if (true) {  
    return "always this";  
} else {  
    return "never reached"; // Unreachable  
}
```

4. Platform-specific kod

```
if (process.platform === 'win32') {  
    // Windows-specifik kod  
} else {  
    // Unix-specifik kod - körs inte på Windows-testmiljö  
}
```

5. Async edge cases

- Race conditions
- Timeout-scenarier
- Concurrency issues

6. External dependencies

- Third-party API calls
- Database connections
- File system operations (Om inte mockade)

I våra exempel

Täcks: ☒ Constructor validation ☒ Hash generation ☒ Password comparison ☒ Error cases med giltiga inputs

Kan vara svårt att täcka: ✗ Out of memory vid extremt långa strängar ✗ Hash-algorithm edge cases för alla möjliga inputs ✗ System-level errors

Praktiskt mål

80-100% coverage för business logic är rimligt mål, men:

- 100% är inte alltid möjligt
- 100% är inte alltid meningsfullt
- Fokus bör vara på kritisk kod och edge cases

Viktigt: Code coverage mäter KÖR kod, inte TESTAD kod. Hög coverage garanterar inte kvalitet!

What kind of "asserts"/"expects" can be done in your testing framework?

I Jest (vårt testramverk) finns många typer av expects:

1. Equality Matchers

```
expect(value).toBe(expectedValue)           // Exakt likhet (===)
expect(value).toEqual(expectedValue)         // Deep equality för objekt
expect(value).not.toBe(unexpectedValue)      // Negation
expect(value).toStrictEqual(expectedValue)    // Strict deep equality
```

Användning i våra tester:

```
expect(helper.isValidDay('31')).toBe(true)
expect(ssn.getYear()).toBe('23')
```

2. Truthiness

```
expect(value).toBeTruthy()
expect(value).toBeFalsy()
expect(value).toBeNull()
expect(value).toBeUndefined()
expect(value).toBeDefined()
```

3. Numbers

```
expect(value).toBeGreaterThan(3)
expect(value).toBeGreaterThanOrEqual(3.5)
expect(value).toBeLessThan(5)
expect(value).toBeLessThanOrEqual(4.5)
expect(value).toBeCloseTo(0.3) // För floating point
```

4. Strings

```
expect(string).toMatch(/pattern/)
expect(string).toMatch('substring')
expect(string).toContain('substring')
```

5. Arrays and Iterables

```
expect(array).toContain(item)
expect(array).toContainEqual(item)
expect(array).toHaveLength(number)
```

6. Exceptions/Errors

```
expect(() => func()).toThrow()
expect(() => func()).toThrow(Error)
expect(() => func()).toThrow('error message')
expect(() => func()).toThrow(/pattern/)
```

Användning i våra tester:

```
expect(() => new Password('')).toThrow('Too short password, minimum 12
characters')
expect(() => new SwedishSocialSecurityNumber(badLength, mockHelper))
    .toThrow('To short, must be 11 characters')
```

7. Objects

```
expect(object).toHaveProperty('key')
expect(object).toHaveProperty('key', value)
expect(object).toMatchObject(partialObject)
```

8. Promises (async testing)

```
await expect(promise).resolves.toBe(value)
await expect(promise).rejects.toThrow()
```

9. Mock-specific

```
expect(mockFn).toHaveBeenCalled()
expect(mockFn).toHaveBeenCalledTimes(number)
expect(mockFn).toHaveBeenCalledWith(arg1, arg2)
```

Vilka använder vi i våra tester?

Password tests:

- `.toBe()` - För boolean och string jämförelser

- `.toThrow()` - För att verifiera exceptions

SSNHelper tests:

- `.toBe()` - För boolean returvärden

SwedishSocialSecurityNumber tests:

- `.toThrow()` - För constructor validation
- `.toBe()` - För getter-metoder
- `.not.toThrow()` - För giltiga inputs

Viktigt att välja rätt matcher

- ☒ Använd `.toBe()` för: primitiver (number, string, boolean) ☒ Använd `.toEqual()` för: objekt och arrays
- ☒ Använd `.toThrow()` för: exception testing ☒ Använd `.toBeCloseTo()` för: floating point numbers
-

What kind(s) of Code Coverage is shown in your code coverage tool?
What does the different coverage values mean?

Vårt coverage-verktyg (Jest med Istanbul) visar 4 typer av coverage:

1. Statement Coverage (% Stmts)

Definition: Procentandel av alla statements (kodrader) som körs under tester.

Exempel:

```
const x = 5;           // Statement 1
const y = 10;          // Statement 2
return x + y;          // Statement 3
```

Om alla 3 statements körs: 100% statement coverage.

I våra tester:

File	% Stmts
SwedishSocialSecurityNumber.js	91.42
SSNHelper.js	100
Password.js	94.23

2. Branch Coverage (% Branch)

Definition: Procentandel av alla beslutspunkter (if/else, switch, ternary) där båda vägarna testas.

Exempel:

```
if (password.length >= 12) {    // Branch point
  return true;                  // Branch 1
} else {
  return false;                 // Branch 2
}
```

För 100% branch coverage måste båda `true` och `false` fallet testas.

I våra tester:

File	% Branch
SwedishSocialSecurityNumber.js	77.77
SSNHelper.js	100
Password.js	83.33

3. Function Coverage (% Funcs)

Definition: Procentandel av funktioner/metoder som anropas minst en gång.

Exempel:

```
function foo() { }    // Function 1
function bar() { }    // Function 2
```

Om bara `foo()` anropas: 50% function coverage.

I våra tester:

File	% Funcs
SwedishSocialSecurityNumber.js	100
SSNHelper.js	100
Password.js	100

4. Line Coverage (% Lines)

Definition: Procentandel av körbara kodrader som exekveras.

Skillnad från Statement: En rad kan innehålla flera statements.

Exempel:

```
const x = 5, y = 10;    // En rad, två statements
```

I våra tester:

File	% Lines
SwedishSocialSecurityNumber.js	90.90
SSNHelper.js	100
Password.js	93.87

Vad betyder värdena i våra tester?

SSNHelper: 100% på allt

☒ Alla statements körs ☒ Alla branches testas ☒ Alla funktioner anropas ☒ Alla rader exekveras =
Perfekt täckning!

SwedishSocialSecurityNumber: 77.77% Branch

Betyder att cirka 22% av beslutspunkterna inte testas i båda riktningarna.

- Vissa if/else har bara en väg testad
- Möjliga otestade edge cases

Password: 83.33% Branch

Betyder att cirka 17% av branches saknar täckning.

- Vissa valideringsgrenar kanske inte testas fullt ut

Uncovered Line #s

Verktöget visar också vilka radnummer som INTE täcks:

```
Uncovered Line #s
12-15, 23
```

Betyder att raderna 12, 13, 14, 15 och 23 aldrig körs i testerna.

Vad är viktigast?

Branch Coverage är ofta viktigast eftersom:

- Statement/Line coverage kan vara 100% utan att alla vägar testas
- Branch coverage säkerställer att alla beslut testas i båda riktningarna

Exempel:

```
if (x > 5) {
  console.log("Greater");
}
```

```
}  
// No else
```

- 100% statement coverage med bara `x = 10`
- Men bara 50% branch coverage (bara true-fallet testat)

Målsättning

- **Minimum:** 80% på alla metrics
- **Bra:** 90%+ på alla metrics
- **Excellent:** 95%+ med fokus på kritisk kod
- **Perfekt:** 100% (men inte alltid praktiskt/möjligt)

Why should a single test only have one assert/expect?

Svar: För att varje test ska ha ett tydligt, specifikt syfte och ge meningsfull feedback när det failar.

Huvudskäl

1. Tydlig failure feedback

När ett test failar vill vi veta EXAKT vad som gick fel.

Dåligt exempel (flera asserts):

```
it('validates password', () => {  
  expect(password.length).toBeGreaterThan(12); // Failar här  
  expect(password.containsNumber()).toBe(true); // Körs aldrig!  
  expect(password.isHashed()).toBe(true);      // Körs aldrig!  
});
```

Om första assert failar, körs inte resten → vi missar information.

Bra exempel (en assert per test):

```
it('should have length greater than 12', () => {  
  expect(password.length).toBeGreaterThan(12);  
});  
  
it('should contain number', () => {  
  expect(password.containsNumber()).toBe(true);  
});  
  
it('should be hashed', () => {  
  expect(password.isHashed()).toBe(true);  
});
```

Varje test failar oberoende → vi ser alla problem.

2. Single Responsibility Principle

Varje test testar EN sak:

- Lättare att förstå vad som testas
- Lättare att hitta och fixa bugs
- Tydligare testnamn

3. Bättre testnamn

```
// Svårt att namnge:  
it('validates multiple things', () => { /* ... */ });  
  
// Lätt att namnge:  
it('should throw error for password without numbers', () => { /* ... */ });
```

4. Lättare debugging

När ett test failar:

- Du vet EXAKT vad som är fel från testnamnet
- Ingen gissning om vilket assert som failade

Undantag: Relaterade asserts

Det finns **legitima undantag** när asserts testar samma koncept:

Exempel 1: Getter-metoder (vårt Test 4 för SwedishSocialSecurityNumber)

```
it('4 - getters Should Return Correct Values For Valid SSN', () => {  
  const ssn = new SwedishSocialSecurityNumber(validSSN, mockHelper);  
  expect(ssn.getYear()).toBe('23');  
  expect(ssn.getMonth()).toBe('10');  
  expect(ssn.getDay()).toBe('13');  
  expect(ssn.getSerialNumber()).toBe('2390');  
});
```

Motivering: Alla getters testar SAMMA koncept - att SSN parsas korrekt. Att dela upp skulle vara onödigt.

Exempel 2: Object state

```
it('should create valid person object', () => {  
  const person = new Person('John', 30);  
  expect(person.name).toBe('John');  
});
```

```
    expect(person.age).toBe(30);  
  });
```

Exempel 3: Array innehåll

```
it('should return correct array', () => {  
  const result = getArray();  
  expect(result).toHaveLength(3);  
  expect(result).toContain('a');  
});
```

Riktlinjer

☒ **En assert per test** - som huvudregel ☒ **Undantag OK när** asserts testar samma logiska koncept ☒
Aldrig OK att testa helt olika funktioner i samma test

I våra tester

Password:

- Test 1-9: En assert vardera ☒
- Test 11: En assert ☒

SSNHelper:

- Test 1-6: En assert vardera ☒

SwedishSocialSecurityNumber:

- Test 1-3: En assert vardera ☒
- Test 4: Fyra asserts (getter-metoder) - **Motiverat undantag** ☒

What is the two main purposes of using mocks?

De två huvudsakliga syftena med mocks:

1. Isolation - Testa SUT (System Under Test) isolerat

Syfte: Isolera klassen som testas från dess beroenden.

Varför?

- Vi vill testa ENDAST den klass vi fokuserar på
- Inte blanda in bugs från andra klasser
- Inte vara beroende av andras implementation

Exempel från våra tester:

```
// SwedishSocialSecurityNumber UTAN mock (integration test)
const helper = new SSNHelper(); // Verklig SSNHelper
const ssn = new SwedishSocialSecurityNumber('231013-2390', helper);
// Om testet failar: bug i SwedishSocialSecurityNumber ELLER SSNHelper?

// SwedishSocialSecurityNumber MED mock (unit test)
const mockHelper = {
  isCorrectLength: ssn => ssn.length === 11,
  isCorrectFormat: ssn => /^d{6}-d{4}$/.test(ssn),
  // ... kontrollerat beteende
};
const ssn = new SwedishSocialSecurityNumber('231013-2390', mockHelper);
// Om testet failar: bug i SwedishSocialSecurityNumber!
```

Fördelar: ☒ Tydlig felkälla - vi vet exakt var buggen är ☒ Snabbare tester - ingen verklig kod körs ☒
Oberoende - kan testa även om SSNHelper inte är klar

2. Control - Kontrollera beteende och simulera edge cases

Syfte: Styra vad beroenden returnerar för att testa olika scenarios.

Varför?

- Testa edge cases som är svåra att skapa med verkliga objekt
- Simulera fel och exceptions
- Garantera deterministiskt beteende

Exempel från våra tester:

```
// Test 3 - Simulera Luhn-fel
const mockWrongLuhn = {
  ...mockHelper,
  luhnIsCorrect: () => false // Kontrollerat: returnerar alltid false
};
expect(() => new SwedishSocialSecurityNumber(validSSN, mockWrongLuhn))
  .toThrow("Invalid SSN according to Luhn's algorithm");
```

Fördelar: ☒ Kan testa alla code paths utan att behöva hitta "rätt" testdata ☒ Kan simulera sällsynta fel ☒
Deterministiska tester - samma resultat varje gång

Exempel på svåra scenarios att testa utan mock

1. Network failures:

```
const mockApiClient = {
  fetchData: () => { throw new Error('Network timeout'); }
};
```

2. Database errors:

```
const mockDb = {
  save: () => { throw new Error('Connection lost'); }
};
```

3. Random/Time-dependent kod:

```
const mockRandom = {
  getRandom: () => 0.5 // Alltid samma värde
};
```

4. Edge cases i våra tester:

```
// Test 1 - Whitespace som är svårt att simulera med verklig SSNHelper
const ssnWithWhitespace = ' 231013-2390 ';
// Mock gör att vi kan fokusera på SwedishSocialSecurityNumber's trim-funktion
```

Sammanfattning

Syfte	Vad	Varför	Exempel från våra tester
Isolation	Isolera SUT från beroenden	Testa en klass i taget	Mock SSNHelper när vi testar SwedishSocialSecurityNumber
Control	Kontrollera returvärden	Testa alla edge cases	Mock luhnisCorrect för att returnera false

I våra tester används mocks för

SwedishSocialSecurityNumber.test.js:

- ☒ Isolation: Testar SwedishSocialSecurityNumber oberoende av SSNHelper
- ☒ Control: Kan testa alla valideringsgrenar genom att styra mock-returvärden

Resultat:

- Varje test fokuserar på EN klass
- Alla edge cases kan testas
- Snabba, deterministiska tester

Why is it good if a test for a SUT is independent of the classes that the SUT depends on?

SUT (System Under Test) = Klassen vi testar

Svar: För att få snabba, pålitliga och fokuserade tester som tydligt visar var problem uppstår.

Huvudfördelar

1. Tydlig felkälla

Med beroenden:

```
// Integration test - använder verklig SSNHelper
it('should accept valid SSN', () => {
  const helper = new SSNHelper(); // Verklig klass
  const ssn = new SwedishSocialSecurityNumber('231013-2390', helper);
  expect(() => ssn).not.toThrow();
});
// Om detta failar: Bug i SwedishSocialSecurityNumber? Eller i SSNHelper?
```

Utan beroenden (med mock):

```
// Unit test - mock
it('should accept valid SSN', () => {
  const mockHelper = { /* kontrollerat beteende */ };
  const ssn = new SwedishSocialSecurityNumber('231013-2390', mockHelper);
  expect(() => ssn).not.toThrow();
});
// Om detta failar: Bug i SwedishSocialSecurityNumber! (mockens beteende är känt)
```

Fördel: Vi vet EXAKT var buggen är → snabbare debugging.

2. Snabbare tester

Med beroenden:

```
const helper = new SSNHelper();
// - Skapar objekt
// - Kör verklig validering
// - Eventuellt database/API calls
// - Långsam exekvering
```

Utan beroenden:

```
const mockHelper = {
  isCorrectLength: ssn => ssn.length === 11 // Direkt returvärde
};
// - Ingen objektskapning
```

```
// - Ingen komplex logik
// - Blixtsnabb exekvering
```

Fördel: Tusentals tester kan köras på sekunder → TDD möjligt.

3. Pålitliga/Deterministiska tester

Med beroenden:

```
const helper = new SSNHelper();
// - Beroende av SSNHelper's implementation
// - Om SSNHelper ändras, kan testet faila
// - Svårt att förutse beteende
```

Utan beroenden:

```
const mockHelper = {
  luhnCorrect: () => true // Alltid samma resultat
};
// - Garanterat beteende
// - Testet failar bara om SUT ändras
// - Förutsägbart
```

Fördel: Inga "flaky tests" som failar slumpmässigt.

4. Kan testa även om beroenden inte är klara

Med beroenden:

```
// Måste vänta tills SSNHelper är klar och testad
const helper = new SSNHelper(); // Kanske inte implementerad än
```

Utan beroenden:

```
// Kan börja testa direkt
const mockHelper = {
  isCorrectLength: ssn => true // Simulated beteende
};
// Kan testa SwedishSocialSecurityNumber innan SSNHelper finns!
```

Fördel: Parallell utveckling möjlig → snabbare utvecklingscykel.

5. Fokus på EN sak i taget

Med beroenden:

```
// Testar både SwedishSocialSecurityNumber OCH SSNHelper
const helper = new SSNHelper();
const ssn = new SwedishSocialSecurityNumber('231013-2390', helper);
```

Utan beroenden:

```
// Testar ENDAST SwedishSocialSecurityNumber
const mockHelper = { /* ... */ };
const ssn = new SwedishSocialSecurityNumber('231013-2390', mockHelper);
```

Fördel: Följer Single Responsibility Principle för tester.

6. Testa alla edge cases enkelt**Med beroenden:**

```
// Svårt att få SSNHelper att returnera specifika fel
const helper = new SSNHelper();
// Hur får vi Luhn att faila för ett specifikt värde?
```

Utan beroenden:

```
// Enkelt att simulera alla scenarios
const mockWrongLuhn = {
  ...mockHelper,
  luhnIsCorrect: () => false // Tvingar ett specifikt fel
};
```

Fördel: 100% code coverage lättare att uppnå.

Exempel från våra tester

SwedishSocialSecurityNumber MED mock (Unit Test)

```
describe('SwedishSocialSecurityNumber', () => {
  const mockHelper = {
    isCorrectLength: ssn => ssn.length === 11,
    isCorrectFormat: ssn => /^\\d{6}-\\d{4}$/.test(ssn),
    isValidMonth: month => parseInt(month, 10) >= 1 && parseInt(month, 10) <=
12,
    isValidDay: day => parseInt(day, 10) >= 1 && parseInt(day, 10) <= 31,
```

```

    luhnCorrect: ssn => ssn === validSSN
  };

  it('1 - should accept valid SSN with whitespace', () => {
    const ssnWithWhitespace = ' 231013-2390 ';
    expect(() => new SwedishSocialSecurityNumber(ssnWithWhitespace,
mockHelper))
      .not.toThrow();
  });
  // Testar ENDAST SwedishSocialSecurityNumber's trim-funktion
});

```

Resultat: ☒ Om testet failar → problem i SwedishSocialSecurityNumber ☒ Snabbt test ☒ Kan testa whitespace-hantering isolerat ☒ Inget beroende av SSNHelper's implementation

SSNHelper UTAN mock (Unit Test för hjälpklassen)

```

describe('SSNHelper isValidDay', () => {
  it('3- should return false for day 00', () => {
    const helper = new SSNHelper();
    expect(helper.isValidDay('00')).toBe(false);
  });
});

```

Resultat: ☒ Testar SSNHelper isolerat ☒ Inga beroenden att mocka (hjälpklass utan dependencies)

Sammanfattning

Fördel	Med beroenden	Utan beroenden (mock)
Felkälla	Oklar	Tydlig
Hastighet	Långsam	Snabb
Pålitlighet	Beroende av andra	Deterministisk
Utveckling	Måste vänta	Kan börja direkt
Fokus	Flera klasser	En klass
Edge cases	Svåra	Enkla

Bästa praxis:

- Unit tests → Mocka beroenden
- Integration tests → Använd verkliga beroenden
- Båda behövs för full testning!

Can all bugs be found by testing?

Svar: NEJ, alla buggar kan INTE hittas genom testning.

Varför inte?

1. Oändligt antal möjliga inputs

Exempel:

```
function add(a, b) {  
  return a + b;  
}
```

Omöjligt att testa:

- Alla kombinationer av tal
- Alla edge cases: Infinity, -Infinity, NaN
- Alla datatyper: number, string, null, undefined, object

Vi kan bara testa ett urval.

2. Komplexitet växer exponentiellt

```
function validate(name, age, email, password) {  
  // Varje parameter har många möjliga värden  
  // Kombinationer = n^4 där n = antal möjliga värden per parameter  
}
```

Med bara 10 möjliga värden per parameter: $10^4 = 10,000$ kombinationer!

Omöjligt att testa alla kombinationer.

3. Concurrency och timing-buggar

```
// Race condition - beroende av timing  
let counter = 0;  
async function increment() {  
  const temp = counter;  
  await someAsyncOperation();  
  counter = temp + 1;  
}
```

Svårt att reproducera:

- Beroende av exakt timing
- Kan faila slumpmässigt
- Olika beteende i olika miljöer

4. Integration och system-buggar

Exempel:

- Fungerar i test-miljö men inte i produktion
- Fungerar med mock men inte med verklig databas
- Fungerar lokalt men inte i molnet
- Fungerar för små dataset men inte stora

5. Logiska fel i spec/krav

```
// Krav: "Password måste vara minst 12 tecken"
function validatePassword(pwd) {
    return pwd.length >= 12; // Implementerat korrekt enligt spec
}

// Men kravet var FEL - borde vara minst 8 tecken!
```

Tester validerar implementation mot spec, inte spec mot verklighet.

6. Edge cases vi inte tänker på

Exempel från våra tester:

```
// Vi testade:
expect(helper.isValidDay('00')).toBe(false); // Dag 00
expect(helper.isValidDay('31')).toBe(true); // Dag 31

// Men missade kanske:
// Dag 29 februari på icke-skottår
// Dag 31 april (april har bara 30 dagar)
// Negativa dagar: '-01'
// Bokstäver: 'AB'
```

7. Environment-specifika buggar

- Browser-specifika problem (Chrome vs Firefox)
- OS-specifika problem (Windows vs Linux)
- Hardware-specifika problem (arkitektur, minne)

8. Säkerhetsbrister

- SQL injection
- XSS attacks
- Buffer overflows
- Dessa kräver specialiserad säkerhetstestning, inte bara unit tests

9. Performance-problem

```
function slowFunction(n) {  
  if (n === 1000000) {  
    // Blir extremt långsam  
  }  
}
```

Unit test med små värden hittar inte detta.

Vad kan vi göra?

Testa strategiskt

1. **Boundary values** - Testa gränsvärden

```
// Min, min-1, min+1, max-1, max, max+1
```

2. **Equivalence partitioning** - Gruppera liknande inputs

```
// Istället för alla tal 1-100, testa: 1, 50, 100
```

3. **Edge cases** - Fokusera på ovanliga fall

```
// null, undefined, '', 0, negative, very large
```

4. **Code coverage** - Sikta på hög coverage

- Visar vad som INTE testas
- Men 100% coverage ≠ inga buggar!

5. **Different test types:**

- Unit tests
- Integration tests
- System tests
- Performance tests
- Security tests
- User acceptance tests

6. **Code review** - Mänsklig granskning hittar logiska fel

7. **Static analysis** - Verktyg som hittar potentiella problem

8. **Formal verification** - Matematiska bevis (för kritiska system)

I våra exempel

Vad vi hittade: ☒ 10 Password-buggar ☒ 7 SSNHelper-buggar ☒ 4 SwedishSocialSecurityNumber-buggar

Vad vi kanske missade: ☒ Unicode-hantering ☒ Mycket långa inputs (memory issues) ☒ Concurrency-problem om flera trådar ☒ Performance med miljontals passwords ☒ Säkerhetsaspekter (timing attacks på hash-jämförelse)

Viktig princip

Testing kan bara visa närvaron av buggar, aldrig deras frånvaro.

- Edsger Dijkstra

Sammanfattning

Kan testas	Svårt att testa
☒ Normala use cases	☒ Alla möjliga inputs
☒ Kända edge cases	☒ Okända edge cases
☒ Funktionella krav	☒ Implicit förväntningar
☒ Deterministiskt beteende	☒ Race conditions
☒ Unit-nivå logik	☒ System-nivå integration

Slutsats: Testning är essentiellt men inte tillräckligt. Kombinera med:

- Code review
- Static analysis
- Monitoring i produktion
- User feedback
- Kontinuerlig förbättring

Do all tests need asserts/expects?

Svar: Tekniskt NEJ, men praktiskt JA - nästan alla tester behöver asserts/expects.

När behövs INTE asserts?

1. Smoke tests / Compilation tests

```
it('should import without errors', () => {  
  const { Password } = require('./Password');  
  // Inget assert - testet passerar om importen inte kastar fel  
});
```


Syfte: Verifiera att koden kan laddas och kompileras.

2. Exception tests (implicit assert)

```
it('should not throw exception for valid password', () => {  
  new Password('validPassword123');  
  // Inget explicit assert  
  // Om exception kastas → testet failar  
  // Om ingen exception → testet passerar  
});
```

Men bättre är explicit:

```
it('should not throw exception for valid password', () => {  
  expect(() => new Password('validPassword123')).not.toThrow();  
  // Tydligare intention  
});
```

3. Setup/Teardown verification

```
beforeEach(() => {  
  database.connect();  
  // Inget assert - men bra att verifiera  
});
```

4. Test helpers som bara kör kod

```
it('executes without errors', () => {  
  someComplexOperation();  
  // Testar att operationen inte kraschar  
});
```

Varför nästan alla tester BORDE ha asserts

1. Tydlig intention

```
// Vad testar detta?  
it('creates password', () => {  
  const pwd = new Password('test123456789');  
});  
  
// Tydligt!
```

```
it('creates password', () => {  
  const pwd = new Password('test123456789');  
  expect(pwd.getPasswordHash()).toBeDefined();  
});
```

2. Förhindrar "false positives"

```
// Dåligt - passerar även om getYear() returnerar fel  
it('getYear returns value', () => {  
  const ssn = new SwedishSocialSecurityNumber('231013-2390', mockHelper);  
  ssn.getYear(); // Inget assert - testet passerar alltid!  
});  
  
// Bra - failar om getYear() returnerar fel värde  
it('getYear returns value', () => {  
  const ssn = new SwedishSocialSecurityNumber('231013-2390', mockHelper);  
  expect(ssn.getYear()).toBe('23');  
});
```

3. Dokumenterar förväntningar

Assert är levande dokumentation:

```
it('password hash should be different from plain text', () => {  
  const pwd = new Password('secret123456');  
  expect(pwd.getPasswordHash()).not.toBe('secret123456');  
  // Tydligt: hash != plain text  
});
```

Exempel från våra tester

Alla våra tester HAR asserts

Password (10 tester):

```
it('1 - constructor Should Not Store Plain Password As Hash', () => {  
  const pwd = new Password('validPassword1');  
  expect(pwd.getPasswordHash()).not.toBe('validPassword1'); // ☒ Assert  
});
```

SSNHelper (6 tester):

```
it('1 - isCorrectLength Should Return False For SSN With Length 12', () => {  
  const helper = new SSNHelper();
```

```
expect(helper.isCorrectLength('850215-12345')).toBe(false); // ☒ Assert
});
```

SwedishSocialSecurityNumber (4 tester):

```
it('1 - constructor Should Not Throw Exception For Valid SSN With Whitespace', ()
=> {
  expect(() =>
    new SwedishSocialSecurityNumber(ssnWithWhitespace, mockHelper)
  ).not.toThrow(); // ☒ Assert
});
```

När kan man skippa assert?

Jest har en warning

Your test suite must contain at least one assertion

Men vissa ramverk tillåter:

Snapshot testing (Jest)

```
it('renders correctly', () => {
  const tree = renderer.create(<Component />).toJSON();
  expect(tree).toMatchSnapshot(); // Tekniskt ett assert, men annorlunda
});
```

Property-based testing

```
it('always returns positive', () => {
  fc.assert(
    fc.property(fc.integer(), (n) => {
      return Math.abs(n) >= 0; // Implicit assert
    })
  );
});
```

Best practices

☒ **Använd alltid asserts** för att:

- Verifiera förväntade resultat

- Dokumentera beteende
- Förhindra false positives

✗ Skippa bara asserts när:

- Du testar att kod inte kraschar (och är explicit om detta)
- Det är ett smoke test
- Du använder specialiserade testverktyg

Sammanfattning

Scenario	Assert?	Varför?
Funktionellt test	☒ Ja	Verifiera output
Exception test	☒ Ja	Explicit är bättre
Getter test	☒ Ja	Verifiera värde
Smoke test	✗ Kanske inte	Bara att koden laddar
Setup test	✗ Kanske inte	Bara exekvering

Rekommendation: 99% av dina tester borde ha minst ett assert/expect.

Can we with testing prove that we are 100% bug free?

Svar: NEJ - testning kan ALDRIG bevisa att kod är 100% buggfri.

Grundläggande orsaker

1. Oändligt antal test cases

Exempel:

```
function add(a, b) {  
  return a + b;  
}
```

För att bevisa att `add()` är buggfri måste vi testa:

- ∞ kombinationer av a och b
- Alla datatyper
- Alla edge cases

Omöjligt!

2. Halting Problem

Alan Turing bevisade matematiskt att det är **omöjligt** att skriva ett program som kan avgöra om ett annat program kommer:

- Terminera (sluta köra)
- Köra för alltid (infinite loop)

Om vi inte ens kan bevisa detta, hur kan vi bevisa frånvaro av buggar?

3. Dijkstra's citat:

"Program testing can be used to show the presence of bugs, but never to show their absence!"

- Edsger Dijkstra

Testning kan:

- ☒ Visa att buggar FINNS (när test failar)
- ☐ INTE visa att buggar INTE finns (när test passerar)

Varför inte?

1. Vi kan missa edge cases

Vårt exempel:

```
// Vi testade:
expect(helper.isValidDay('00')).toBe(false);
expect(helper.isValidDay('31')).toBe(true);

// Men vi missade kanske:
// - Negativa tal: '-5'
// - Decimaltal: '15.5'
// - Bokstäver: 'AB'
// - Mycket stora tal: '9999'
// - null, undefined
// - Objects, arrays
```

Även med 100% code coverage kan buggar finnas i scenarier vi inte tänkt på.

2. Hidden bugs i kombinationer

```
function complex(a, b, c, d) {
  // Bugg uppstår bara när:
  // a = 5 OCH b = -1 OCH c = 0 OCH d = 'test'
}
```

Med 10 möjliga värden per parameter: $10^4 = 10,000$ kombinationer!

Vi kanske testar 50 kombinationer och missar den buggiga.

3. Miljö-specifika buggar

```
// Fungerar i test
const result = functionThatWorks();

// Men failar i produktion p.g.a.:
// - Olika OS
// - Olika timezone
// - Större dataset
// - Concurrency
// - Network latency
```

4. Logiska fel i specifikation

```
// Spec: "Lösenord måste vara minst 12 tecken"
function validatePassword(pwd) {
  return pwd.length >= 12; // ☒ Korrekt enligt spec
}

// Men spec var FEL - borde vara minst 8!
// Test passerar, men systemet är felaktigt
```

Tester validerar implementation mot spec, inte spec mot verklighet.

5. Code coverage är inte perfekt

100% code coverage betyder:

- Alla rader körs
- Alla branches testas

Men betyder INTE:

- Alla möjliga inputs testade
- Alla edge cases täckta
- Korrekt logik

Exempel:

```
function divide(a, b) {
  return a / b; // 100% coverage med divide(6, 2)
}

// Men vad händer med divide(5, 0)? Division by zero!
```

Vad kan vi göra istället?

1. Minska risk för buggar

Bra testning:

- ☒ 80-100% code coverage
- ☒ Boundary value testing
- ☒ Edge case testing
- ☒ Integration testing

Andra metoder:

- Code review
- Static analysis (linters, type checkers)
- Formal verification (för kritiska system)
- Runtime monitoring

2. Kontinuerlig förbättring

Utveckla → Testa → Deploy → Monitor → Hitta bug → Fixa → Lägg till test → Repeat

- Lägg till test för varje bug som hittas
- Förbättra testsviten kontinuerligt

3. Olika typer av testning

Test typ	Syfte
Unit tests	Testa individuella funktioner
Integration tests	Testa samverkan mellan komponenter
System tests	Testa hela systemet
Performance tests	Testa hastighet och skalbarhet
Security tests	Testa säkerhet
User acceptance tests	Testa användbarhet

Ingen typ hittar alla buggar ensam!

4. Defense in depth

Flera lager av försvar:

1. **Development:** Type systems (TypeScript), linters
2. **Testing:** Unit, integration, system tests
3. **Review:** Code review, pair programming

4. **Deployment:** Staging environments, canary releases
5. **Production:** Monitoring, alerting, logging
6. **Response:** Quick rollback, incident response

Vad visar våra tester?

Vad vi VET:

- ☒ 10 Password-buggar hittade och fixade
- ☒ 7 SSNHelper-buggar hittade och fixade
- ☒ 4 SwedishSocialSecurityNumber-buggar hittade och fixade
- ☒ 100% code coverage för många klasser

Vad vi INTE VET:

- ☒ Finns det fler buggar? **Troligen ja!**
- ☒ Fungerar det med alla inputs? **Omöjligt att veta**
- ☒ Fungerar det i produktion? **Måste testa där också**
- ☒ Är specen korrekt? **Kräver användartester**

Matematisk analogi

Testing är som att söka efter mynt i en oändlig sandlåda:

 Sandlåda (alla möjliga inputs)

 Mynt (buggar)

Du gräver 1000 gånger och hittar 5 mynt.

Fråga: Finns det fler mynt?

Svar: Troligen! Men du kan aldrig vara säker.

Slutsats

Påstående	Sant/Falskt
"Alla tester passerar" → inga buggar	☒ Falskt
"100% code coverage" → inga buggar	☒ Falskt
"Testning hittar buggar"	☒ Sant
"Testning minskar buggar"	☒ Sant
"Testning bevisar frånvaro av buggar"	☒ Falskt

Vi kan:

- ☒ Öka konfidensen i koden
- ☒ Minska antalet buggar
- ☒ Hitta kända typer av buggar

- ☒ Förhindra regression

Vi kan INTE:

- ☒ Bevisa att inga buggar finns
- ☒ Testa alla möjliga inputs
- ☒ Garantera 100% korrekthet

Bästa strategi:

1. Skriv bra tester (hög coverage, edge cases)
2. Kombinera med andra metoder (review, static analysis)
3. Monitera produktion
4. Var ödmjuk - anta att buggar finns och planera för dem
5. Kontinuerlig förbättring

Acceptera: Perfect är fienden till good. Sikta på hög kvalitet, inte omöjlig perfektion.